

RegRadar Frontend Specification Document

Contents

RegRadar — Frontend Specification Document	1
1. Design System	1
2. Screen Inventory (for context)	6
3. Frontend Architecture & Data Flow	6
4. Third-Party & Integration Specification	7

RegRadar — Frontend Specification Document

Version 1.0 | Design System + Third-Party Integration Spec

Scope note

The RegRadar PRD scopes V1 as API-first with no UI (NG3), and names a React dashboard as a V2 stretch goal. This document specifies that dashboard fully now, so the moment V2 work starts there's a ready-to-build design system and integration contract rather than a blank page. Everything here is written to sit on top of the API contract, database schema, and role/permission model already defined in the Technical Architecture and Security & Access documents — nothing here contradicts those; it's the visual and integration layer on top of them.

1. Design System

1.1 Design principles

Four rules guide every screen: **legible over decorative** — this is a tool compliance professionals scan under time pressure, not a marketing site, so density and clarity beat visual flourish. **Risk is always color-coded consistently** — the Low/Medium/High/Critical scale (already central to the product) uses the same four colors everywhere it appears, with no exceptions, so a user builds instant pattern recognition. **Every destructive or high-consequence action confirms itself** — revoking an API key, deleting a webhook, changing monitored sources — matching the “no silent write access” posture from the Security document. **The system works at data density** — filings tables, obligation lists, and metric dashboards need to hold a lot of information per screen without feeling cluttered, so spacing and type scale are tuned for scanability, not whitespace for its own sake.

1.2 Color palette

All colors are defined as a base scale (50 = lightest, 900 = darkest) so any future component can borrow a consistent shade rather than inventing new ones.

Brand / Primary — Indigo

Token	Hex	Typical use
primary-50	#EEF2FF	Selected-row tint, subtle highlight backgrounds
primary-100	#E0E7FF	Active nav item background
primary-500	#6366F1	Links, focus rings (on dark surfaces)
primary-600	#4F46E5	Primary buttons, active states, brand accent — the main brand color
primary-700	#4338CA	Primary button hover/pressed
primary-900	#312E81	High-contrast text on light brand tints

Neutral — Slate (used for text, borders, backgrounds — the workhorse of the whole UI)

Token	Hex	Typical use
slate-50	#F8FAFC	Page background, table header background
slate-100	#F1F5F9	Hover background for rows/list items
slate-200	#E2E8F0	Default borders, dividers
slate-300	#CBD5E1	Hover borders, input hover state
slate-400	#94A3B8	Placeholder text, disabled text, icons (inactive)
slate-500	#64748B	Secondary/meta text (timestamps, labels)
slate-600	#475569	Body text on secondary surfaces
slate-700	#334155	Nav item text, secondary button text
slate-900	#0F172A	Primary body text, headings

Risk-level semantics — the single most important color mapping in the product; used identically in badges, table rows, chart lines, and email/Slack briefs

Risk	Text/Icon	Background tint	Border
Low	#16A34A	#F0FDF4	#BBF7D0
Medium	#D97706	#FFFBE8	#FDE68A
High	#EA580C	#FFF7ED	#FED7AA
Critical	#DC2626	#FEF2F2	#FECACA

Domain tags (financial / clinical / environmental / other — distinct from risk, never reuse risk colors here to avoid the two systems blurring together)

Domain	Text	Background
Financial	#2563EB	#EFF6FF
Clinical	#7C3AED	#F5F3FF
Environmental	#0D9488	#F0FDF4
Other	#64748B	#F8FAFC

Semantic system colors (distinct from risk colors — used for form validation, toasts, and system banners, not filing risk)

Purpose	Hex
Success	#16A34A

Purpose	Hex
Warning	#D97706
Error	#DC2626
Info	#2563EB

Dark mode variant (secondary priority — light mode is the default for this persona set, but engineers on the team will want dark mode)

Token	Light	Dark
App background	#F8FAFC	#0F172A
Surface/card	FFFFFF	#1E293B
Border	#E2E8F0	#334155
Text primary	#0F172A	#F1F5F9
Text secondary	#475569	#94A3B8
Primary brand	#4F46E5	#6366F1 (brightened for dark backgrounds)

Risk, domain, and semantic colors keep the same hex values in dark mode but sit on #1E293B-family surfaces rather than white — verified for contrast in Section 1.7.

1.3 Typography

Typeface: Inter for all UI text (clean at small sizes, excellent number/tabular-figure support, which matters for a table-heavy product). Fallback stack: -apple-system, "Segoe UI", Roboto, sans-serif.

Monospace: IBM Plex Mono, used only for API keys, webhook IDs, filing accession numbers, and raw JSON/log content. Fallback: ui-monospace, Menlo, monospace.

Style	Size	Weight	Line height	Used for
Display	32px / 2rem	700	1.2	Rare — empty-state hero text only
H1	28px / 1.75rem	700	1.25	Page title (e.g., "Filings")
H2	22px / 1.375rem	600	1.3	Section headers within a page
H3	18px / 1.125rem	600	1.35	Card titles, modal titles
Body Large	16px / 1rem	400	1.5	Executive brief text, key readouts
Body (default)	14px / 0.875rem	400	1.5	Table cells, form inputs, general UI text
Small	12px / 0.75rem	500	1.4	Labels, badges, timestamps, table headers
Mono	13px / 0.8125rem	400	1.5	API keys, IDs, code

Only four font weights are used anywhere in the product — 400, 500, 600, 700 — to keep the type system predictable. Numbers in tables and metric cards use tabular figures (fixed-width digits) so columns of numbers align vertically.

1.4 Spacing & layout

Base unit: 4px. Every margin, padding, and gap value in the product is a multiple of this unit.

Token	Value
xs	4px
sm	8px
md	12px
base	16px
lg	24px
xl	32px
2xl	48px
3xl	64px

Radius: sm 6px (badges, small controls) · md 8px (buttons, inputs) · lg 12px (cards) · xl 16px (modals) · full 9999px (pills, avatars).

Shadow: sm 0 1px 2px rgba(15,23,42,0.06) (resting cards) · md 0 4px 12px rgba(15,23,42,0.08) (hover, dropdowns) · lg 0 12px 32px rgba(15,23,42,0.12) (modals).

Breakpoints: sm 640px · md 768px · lg 1024px · xl 1280px · 2xl 1536px. The product is designed desktop-first (compliance work happens at a desk) with tablet support down to md; below md the sidebar collapses to a bottom nav and tables switch to a stacked card layout per row.

Page layout: Sidebar 240px expanded / 64px collapsed, fixed left. Topbar 56px fixed top, containing page title/breadcrumb, global search, and account menu. Content area max width 1440px, centered, 32px outer padding on desktop / 16px on mobile. Underlying grid: 12 columns, 24px gutters on desktop.

1.5 Components

Buttons

Variant	Background	Text	Border	Use
Primary	slate-900→primary-600 fill	white	none	The one main action per screen (e.g., “Register Webhook”)
Secondary	white	slate-900	slate-200, 1px	Everything else that isn’t primary or destructive
Destructive	error red #DC2626	white	none	Revoke key, delete webhook, disable source
Ghost	transparent	slate-700	none	Low-emphasis actions inside tables/toolbars

Sizes: sm 32px height / 13px text, md 40px height / 14px text (default), lg 48px height / 16px text. States: hover darkens fill one step (or adds slate-50 background for secondary/ghost); focus adds a 2px primary-500 ring with 2px offset; disabled drops to 40% opacity with no pointer cursor; loading replaces the label with a spinner and disables interaction without changing the button’s size (prevents layout shift).

Inputs

Default height 40px, 12px horizontal padding, 1px slate-200 border, 8px radius, 14px body text. States: hover → slate-300 border; focus → primary-500 border plus a 3px primary-100 ring; error → red-600 border with a red-50 background tint and a 12px red-600 message beneath; disabled → slate-50 background, slate-400 text, not-allowed cursor. Labels sit above the field (13px medium, slate-700, 6px margin below); helper or error text sits below (4px margin above).

The global/table search input uses a leading 16px search icon in slate-400. The dedicated RAG search page (Section 3) uses a larger 56px-height, 12px-radius variant to signal it's the primary action on that page, not a filter.

Cards

Base card: white surface, 1px slate-200 border, 12px radius, 20px padding, sm shadow. Clickable cards (e.g., a filing card) add a hover state: md shadow, border shifts to slate-300, 150ms transition.

Filing card layout: top row holds the domain tag and risk badge plus a relative timestamp, right-aligned; middle row is the entity name (H3) and filing type (small, slate-500); bottom is a two-line-clamped excerpt of the executive brief in Body/slate-600.

Metric card layout (used on the Metrics & Cost screen): a small uppercase slate-500 label, a large (28px, bold, slate-900) value, and an optional trend indicator — a small colored arrow with a percentage, green for improving, red for regressing, calibrated per metric (e.g., rising cost-per-filing is a regression; rising faithfulness score is an improvement).

Modals

Overlay: `rgba(15,23,42,0.5)` with a slight backdrop blur. Container: white, 16px radius, lg shadow, width 480px (sm) / 640px (md, default) / 800px (lg) depending on content. 24px internal padding. Header holds an H3 title and a close icon button top-right, with a 1px slate-200 divider beneath it. Footer is a right-aligned button group — Cancel (secondary) plus a Confirm action (primary for neutral actions, destructive-red for anything matching the Security document's "confirm before destructive action" rule) — separated from the body by another 1px divider. Entrance animation: fade + scale-in over 150ms.

Badges & tags

Pill-shaped (full radius), 4px/10px padding, 12px semibold text. Risk badges use the risk palette from 1.2 exactly; domain tags use the domain palette; pipeline-status badges (processing / complete / failed) reuse the semantic info/success/error colors respectively, so a user never has to learn a fifth color system.

Data tables (the single most-used component in the product — the Filings List is the default landing screen)

56px row height, slate-50 header background with 12px uppercase slate-500 semibold labels, header row sticky on scroll. Cells: 16px horizontal / 12px vertical padding. Rows are separated by a 1px slate-100 bottom border rather than full grid lines, and highlight slate-50 on hover — this keeps a data-dense table feeling calm rather than gridded and heavy. Sortable column headers reveal a chevron on hover/active-sort. Empty states center an icon, a message, and an optional call-to-action with 64px vertical padding. Pagination sits bottom-right ("Showing 1-20 of 340") with 40px-tall controls.

Navigation

Sidebar: white background, 1px slate-200 right border (not a dark sidebar — keeps the whole product on one light, calm surface). Active item: primary-50 background, primary-600 text, 3px primary-600 left accent bar. Inactive: slate-600 text; hover: slate-50 background. Each item has an 18px leading icon.

Toasts & banners

Toasts: bottom-right stack, 360px wide, 10px radius, md shadow, 4px colored left border matching the semantic type, icon plus message plus optional action link. Auto-dismiss after 5 seconds, except errors, which require manual dismissal so a failure is never missed. Banners (page-level, e.g., "Slack

delivery has been failing for this workspace”) span the content area width, use a tinted background matching severity, and include an icon, message, and dismiss/action button.

1.6 Iconography

One icon set throughout (an outline/stroke style such as Lucide, 1.5px stroke weight, available at 16/20/24px), never mixed with a second style. Filled icons are reserved for a single specific case — the risk badge dot — to visually distinguish “status” from “action” icons.

1.7 Accessibility

Every text/background pairing in this document meets WCAG AA (4.5:1 for body text, 3:1 for large text and UI components). Spot-checked examples: slate-900 on white is ~16:1; slate-600 on white is ~7.5:1; risk-badge text-on-tint pairs (e.g., Critical red-600 #DC2626 on red-50 #FEF2F2) sit at ~5.9:1. Every interactive element gets a visible focus ring (2px primary-500, 2px offset) — an outline is never removed without an equally visible replacement. Minimum interactive hit area is 40x40px, enforced through padding even where the visible icon is smaller (important in dense table rows, where icon-only action buttons are common).

2. Screen Inventory (for context)

A brief map of the core screens this design system needs to support, so the tokens above aren’t abstract:

Filings List (default landing) — the data table described in 1.5, filterable by domain, risk level, source, and date range, with the global search bar. **Filing Detail** — persona-tabbed briefs, the full obligations/deadlines list, risk score, similar past filings, a link to the original document, and a delivery log for that filing. **Ask RegRadar** — the natural-language search screen (RAG Q&A) for the Legal Counsel persona. **Webhooks** — list, create/edit modal, and a per-webhook delivery log. **API Keys** — masked list, a show-once creation modal, and a destructive-confirm revoke modal. **Metrics & Cost** — Admin/Eng-Lead-only dashboard of the metric cards from 1.5 plus trend charts. **Source Configuration** — Admin-only toggles for which regulators/domains are monitored. **Settings** — org profile, Slack connection, notification preferences.

3. Frontend Architecture & Data Flow

Recommended stack: React + TypeScript on Vite, TanStack Query for server-state fetching/caching (it gives retry and stale-while-revalidate behavior for free, both referenced below), Tailwind CSS configured directly from the design tokens in Section 1 (so “primary-600” in code and in this document are the literal same value), shadcn/ui as a headless component base customized to match Section 1.5 rather than used at its defaults, and React Router for navigation.

Auth/session model: Per the Security & Access Document, RegRadar’s V1 API is authenticated with API keys, which are not meant to live in a browser. The dashboard therefore introduces a session layer on top of that model rather than replacing it: a person logs in via SSO (Section 2 of the Security document), which sets an `HttpOnly`, Secure session cookie. The frontend never reads or stores a raw API key. Every request to RegRadar’s backend automatically includes this cookie; the backend resolves the session to an organization and a role and scopes every response accordingly — the frontend doesn’t independently decide what a CCO-role user can see, it renders whatever the backend already filtered, and additionally hides controls (like “Register Webhook”) that the current role can’t use, as a UX convenience, not a security boundary. A recommended addition to the API contract to support this cleanly: `GET /v1/me`, returning the current session’s role and organization, which the frontend calls once on load to drive navigation and permission-aware UI.

Standard error contract: every backend error response follows one shape — { "error": { "code": "...", "message": "...", "request_id": "..." } }. The frontend keeps a lookup table mapping known code values to user-facing copy and an appropriate UI (inline field error vs. toast vs. full banner); any unrecognized code falls back to a generic “something went wrong, we’ve logged it” banner that still surfaces the request_id, so a support conversation can reference it precisely.

Loading & retry behavior: TanStack Query’s defaults are used almost unmodified — one automatic retry with exponential backoff on network-level failures, no retry on 4xx responses (a bad request won’t fix itself by repeating it), and stale-while-revalidate caching for read-heavy views like the Filings List so navigating back to it feels instant while a fresh copy loads in the background. Nothing security-sensitive uses optimistic UI: creating a webhook, revoking a key, or changing source configuration always waits for the server’s confirmed response before updating the screen, matching the “no assumed success” posture from the Security document’s error-handling guide.

Real-time updates: the Filing Detail screen subscribes directly to Supabase Realtime on the filings table, filtered to that one filing’s ID, so a status of “processing” flips to “complete” without the user refreshing or the frontend polling. This is the one case where the browser talks to a service other than RegRadar’s own backend, and it’s safe specifically because it’s read-only and scoped by the same row-level security rules defined in the Security & Access Document — the Realtime connection uses Supabase’s public anon key, which grants no access beyond what RLS already allows for that authenticated session.

4. Third-Party & Integration Specification

For every external service, this section states what it does, exactly which endpoints are called, what’s sent, what’s expected back, how it authenticates, and — critically for a frontend spec — **whether the browser ever talks to it directly, or whether it’s backend-only**. As a general rule: the browser talks to RegRadar’s own backend and, for the one real-time case above, to Supabase. Every AI provider, data source, and delivery service is backend-only, so their credentials never reach the browser.

4.1 RegRadar Backend API — *called directly by the frontend*

What it is: RegRadar’s own REST API — the single source of truth the dashboard is built against. Full endpoint list and schemas live in the Technical Architecture Document; this section only restates the integration contract from the frontend’s point of view.

Auth: Session cookie (Section 3), not a raw API key.

Example call — loading the Filings List:

```
GET /v1/filings?domain=financial&risk=high&since=2026-01-01&page=1
Cookie: regradar_session=<httponly cookie>
```

// Response 200

```
{
  "data": [
    {
      "id": "flg_8f2a...",
      "entity_name": "Acme Capital",
      "filing_type": "8-K",
      "domain": "financial",
      "risk_level": "high",
      "published_at": "2026-07-15T14:02:00Z",
      "executive_brief": "Acme Capital disclosed a material..."
    }
  ],
  "page": 1,
```

```

    "total": 340
  }

```

Error case: an expired session returns 401 with the standard error envelope; the frontend redirects to the SSO login screen rather than showing a broken page.

4.2 Supabase Realtime — *called directly by the frontend, read-only*

What it is: the managed Postgres/Realtime layer described in the Technical Architecture Document. The frontend uses only its Realtime subscription feature, never direct table writes.

Example call:

```

supabase
  .channel(`filing-${filingId}`)
  .on("postgres_changes",
    { event: "UPDATE", schema: "public",
      table: "filings", filter: `id=eq.${filingId}` },
    (payload) => updateStatus(payload.new.status)
  )
  .subscribe();

```

Auth: Supabase anon (public) key, safe to ship in frontend code because row-level security (Security & Access Document, Section 4) restricts what any given session can actually receive over this channel.

4.3 OpenAI API — *backend-only*

What it does: powers the Analysis Agent's structured extraction (GPT-4o) and the embeddings used for RAG retrieval (text-embedding-3-small). The frontend never calls this directly — doing so would require exposing the OpenAI key in browser code, which the Security document's key-handling rules explicitly rule out.

Endpoints called (by the backend):

POST <https://api.openai.com/v1/chat/completions>
 POST <https://api.openai.com/v1/embeddings>

What's sent (chat/completions, abbreviated):

```

{
  "model": "gpt-4o",
  "response_format": { "type": "json_schema", "json_schema": { "...": "..." } },
  "messages": [
    { "role": "system", "content": "Extract obligations, deadlines..." },
    { "role": "user", "content": "<filing text>" }
  ]
}

```

What's expected back: a JSON object matching the enforced schema — obligations, deadlines, risk_flags, affected_products, key_entities. **Auth:** Authorization: Bearer <OPENAI_API_KEY>. **On failure:** falls back per the tiered-routing and retry rules in the Security document's error handling guide.

4.4 Hugging Face Inference API — *backend-only*

What it does: zero-shot domain classification (facebook/bart-large-mnli) for the Triage Agent, and summarization (facebook/bart-large-cnn, with Granite-13B as the low-tier fallback) for the Summarization Agent.

Endpoints called:

POST https://api-inference.huggingface.co/models/facebook/bart-large-mnli
POST https://api-inference.huggingface.co/models/facebook/bart-large-cnn

What's sent (classification):

```
{
  "inputs": "<filing excerpt>",
  "parameters": {
    "candidate_labels": ["financial", "clinical", "environmental", "other"]
  }
}
```

What's expected back: {"labels": [...], "scores": [...]}, ranked by confidence. **Auth:** Authorization: Bearer <HUGGINGFACE_API_TOKEN>.

4.5 SEC EDGAR — *backend-only*

What it does: the primary ingestion source for SEC filings. **Endpoints called:**

GET https://data.sec.gov/submissions/CIK#####.json
GET https://efits.sec.gov/LATEST/search-index?q=...&forms=8-K

What's sent: no body — a User-Agent header identifying the app and a contact email is required by EDGAR, or requests are rejected (flagged in the Security document's edge-case list). **What's expected back:** JSON listing recent filings with accession numbers and document URLs, which the Ingestion Agent then downloads. **Auth:** none — public API, governed only by the 10 req/sec rate limit already documented.

4.6 FDA RSS Feed — *backend-only*

What it does: ingestion source for FDA guidance and drug-related filings. **Endpoint called:** GET https://www.fda.gov/.../rss.xml (the specific feed URL is a config value, per source_configs in the database schema). **What's sent:** none. **What's expected back:** standard RSS/XML, parsed for new <item> entries since the last poll (tracked via the last_etag/last_polled_at fields already in the schema). **Auth:** none.

4.7 FINRA Feed — *backend-only*

What it does: ingestion source for FINRA rulemaking notices. Same integration shape as the FDA feed — polled on a schedule, no authentication, new items detected against stored state. Specific feed/API details are confirmed during implementation, as FINRA's public interface is more limited than EDGAR's.

4.8 Slack — *backend-only, configured via the frontend*

What it does: delivers Critical/High alerts to a channel. The frontend never talks to Slack directly; instead, a Settings screen lets an Admin paste an Incoming Webhook URL, which is sent to RegRadar's own backend to store and validate (including a "send test message" action that round-trips through the backend, so the frontend never needs Slack credentials at all).

Endpoint called (by the backend, when delivering an alert):

POST https://hooks.slack.com/services/<workspace>/<channel>/<token>

What's sent:

```
{
  "text": "Critical filing: Acme Capital 8-K",
  "blocks": [
    { "type": "section",
      "text": { "type": "mrkdwn",
```

```

    "text": "*Acme Capital* filed an 8-K – Risk: *Critical*" } }
  ]
}

```

What's expected back: HTTP 200 with body ok; any other response is treated as a delivery failure per the Security document's retry/fallback-to-email rule.

4.9 SendGrid — *backend-only*

What it does: sends individual alert emails and the weekly digest. **Endpoint called:** POST <https://api.sendgrid.com/v3/mail/send>. **What's sent:** a JSON payload with personalizations (recipient), from, subject, and HTML/text content built from the persona-specific brief. **What's expected back:** HTTP 202 Accepted on success, with delivery status tracked separately via SendGrid's event webhook (also backend-only). **Auth:** Authorization: Bearer <SENDGRID_API_KEY>.

4.10 AWS S3 — *backend generates access, frontend uses it briefly*

What it does: stores raw filing PDFs. The frontend never holds AWS credentials. When a user clicks "View Original Document" on the Filing Detail screen, the frontend calls RegRadar's own backend (GET /v1/filings/{id}/document-url), which generates a short-lived, pre-signed S3 URL and returns it. The frontend then opens that URL directly (a plain GET, no credentials needed — the signature in the URL itself is the authorization, and it expires after a few minutes).

4.11 LangSmith & Langfuse — *not reachable from the frontend at all*

What they do: prompt versioning, eval tracking, and cost/latency observability, as detailed in the Technical Architecture Document. These are internal engineering tools. The dashboard's Metrics & Cost screen never calls them directly — it calls RegRadar's own GET /v1/metrics endpoint, which the backend populates from data already written to the eval_runs table. This keeps LangSmith/Langfuse credentials and dashboards entirely out of any customer-facing surface, consistent with the Security document's scoping of cost data to the Admin and Eng Lead roles.

4.12 Integration summary table

Service	Purpose	Called by	Auth
RegRadar Backend API	Primary data source for the whole dashboard	Frontend	Session cookie
Supabase Realtime	Live filing status updates	Frontend	Anon key + RLS
OpenAI	Structured extraction, embeddings	Backend only	API key
Hugging Face Inference API	Classification, summarization	Backend only	API key
SEC EDGAR	Filing ingestion	Backend only	None (User-Agent required)
FDA RSS	Filing ingestion	Backend only	None
FINRA Feed	Filing ingestion	Backend only	None
Slack	Alert delivery	Backend only	Webhook URL (customer-configured)
SendGrid	Email delivery	Backend only	API key
AWS S3	Raw PDF storage & retrieval	Backend (frontend uses signed URL)	Pre-signed URL
LangSmith / Langfuse	Internal observability	Backend only, not exposed	API key